

Lời giải trận thứ năm HDU Multi-University 2021

Nguồn gốc: HDU Multi-University 2021 MINIEYE Cup, Contest 5 Solutions

contest/hdu-2021-minieye-05-solutions.pdf

1 1001 Miserable Faith

Với người quen thuộc link-cut tree, có thể nhận ra thao tác 1 đang mô phỏng thao tác access của LCT.

Vì vậy, khi thao tác 1 làm thay đổi cạnh thật và cạnh ảo trong LCT, ta dùng một cây đoạn để cập nhật theo đoạn, đồng thời duy trì đáp án của thao tác 4. Thao tác 2 tương đương với truy vấn tại một điểm. Thao tác 3 tương đương với truy vấn tổng trên một cây con rồi trừ đi phần đáp án tại điểm u .

Độ phức tạp là $O(n \log^2 n)$.

2 1002 String Mod

Ta có thể liệt kê số lần xuất hiện x, y của hai ký tự a, b trong chuỗi:

$$\text{ans}[i][j] = \sum_{x=0}^L \sum_{y=0}^{L-x} [n \mid x-i][n \mid y-j] \binom{L}{x} \binom{L-x}{y} (k-2)^{L-x-y}.$$

Dùng lọc nghiệm bằng căn đơn vị, với w_n là căn bậc n nguyên thủy:

$$\begin{aligned} \text{ans}[i][j] &= \frac{1}{n^2} \sum_{p=0}^{n-1} \sum_{q=0}^{n-1} \sum_{x=0}^L \sum_{y=0}^{L-x} w_n^{px} w_n^{qy} \binom{L}{x} \binom{L-x}{y} (k-2)^{L-x-y} w_n^{-pi} w_n^{-qj} \\ &= \frac{1}{n^2} \sum_{p=0}^{n-1} \sum_{q=0}^{n-1} (w_n^p + w_n^q + k-2)^L w_n^{-pi} w_n^{-qj}. \end{aligned}$$

Đặt

$$A[i][p] = w_n^{-ip}, \quad B[p][q] = \frac{1}{n^2} (w_n^p + w_n^q + k-2)^L, \quad C[q][j] = w_n^{-qj}.$$

Khi đó

$$\text{ans} = A \times B \times C.$$

Độ phức tạp thời gian là $O(n^3 + n^2 \log L)$.

3 1003 VC Is All You Need

Phát biểu tương đương. Trong không gian n chiều, cần tìm giá trị lớn nhất của m sao cho có thể tự chọn m điểm, và với mọi cách tô hai màu các điểm đó, tức mọi trong 2^m phương án tô màu, luôn tồn tại một siêu phẳng $n-1$ chiều tách nghiêm ngặt hai màu.

Kết luận.

$$m_{\max} = n + 1.$$

Do tính đơn điệu, chứng minh chia làm hai phần: chứng minh $m = n + 1$ khả thi, rồi chứng minh $m \geq n + 2$ vô nghiệm.

Với phần thứ nhất, lấy

$$x_0 = (0, 0, \dots, 0),$$

còn x_i là véc-tơ 01 có tọa độ thứ i bằng 1 và các tọa độ khác bằng 0, với $i \in [1, n]$. Xét hàm tuyến tính

$$f_w(x) = w_0 + w_1x_1 + \dots + w_nx_n = w^T[1 \ x]^T.$$

Siêu phẳng phân loại là $f_w(x) \geq 0$.

Với một cách tô màu

$$S = \{(x_i, y_i)\}, \quad y_i \in \{+1, -1\},$$

ta trực tiếp dựng w sao cho $f_w(x_i) = y_i$ với mọi i . Hệ phương trình là

$$w_0 = y_0, \quad w_0 + w_i = y_i \quad (i \in [1, n]),$$

nên luôn giải được.

Với phần thứ hai, thay mỗi x_i bởi $[1 \ x_i]$. Trong không gian $\mathbb{R}^{n+1}$, bất kỳ $n + 2$ điểm nào cũng phụ thuộc tuyến tính. Do đó tồn tại một tổ hợp tuyến tính không tầm thường bằng 0. Chọn một hệ số khác 0 tương ứng với x_0 , ta có thể viết

$$x_0 = \sum_{i=1}^{n+1} a_i x_i.$$

Ta dựng nhãn y_i như sau: nếu $a_i \geq 0$ thì $y_i = 1$, nếu $a_i < 0$ thì $y_i = -1$, và đặt $y_0 = -1$.

Nếu tồn tại w tách được cách tô này, thì với mọi $i \geq 1$ ta có

$$w^T a_i x_i \geq 0.$$

Suy ra

$$w^T x_0 = \sum_{i=1}^{n+1} w^T a_i x_i \geq 0.$$

Nhưng $y_0 = -1$ lại đòi hỏi $f_w(x_0) = w^T x_0 < 0$, mâu thuẫn. Vậy với $n + 2$ điểm luôn có một cách tô không thể tách bởi siêu phẳng $n - 1$ chiều.

4 1004 Another String

Định nghĩa $F[i, j]$ là độ dài lớn nhất của hai chuỗi con bắt đầu lần lượt tại i, j trong S sao cho chúng thỏa k -matching. Nói cách khác, $F[i, j]$ là giá trị lớn nhất của L để

$$S[i, i + L - 1] \quad \text{và} \quad S[j, j + L - 1]$$

thỏa k -matching.

Để tính mọi $F[i, j]$ với $1 \leq i < j \leq n$, ta liệt kê hiệu hai đầu trái $d \in [1, n - 1]$. Với mỗi d , nhờ tính đơn điệu của k -matching, dùng hai con trỏ để tính toàn bộ $F[i, i + d]$ trong thời gian tuyến tính. Vì vậy mọi $F[i, j]$ được tính trong $O(n^2)$.

Tiếp theo xét mọi cách chia chuỗi. Gọi phép chia tại t là

$$A = S[1, t - 1], \quad B = S[t, n].$$

Với phép chia này, đáp án cần thống kê là

$$\sum_{i=1}^{t-1} \sum_{j=t}^n G[i, j], \quad G[i, j] = \min(F[i, j], t - i).$$

Ta liệt kê t từ lớn xuống nhỏ và duy trì đóng góp của G . Vì $G[i, j] \leq t - i$, với mỗi i duy trì mảng count_i để đếm số giá trị $G[i, j]$ đang đóng góp; $\text{count}_i[x]$ là số giá trị có đóng góp bằng x . Đồng thời duy trì max_i , giá trị đóng góp lớn nhất hiện tại của hàng i .

Mỗi khi t giảm, ta cập nhật các mảng đếm và các giá trị lớn nhất để tính thay đổi của đáp án. Trong quá trình này max_i chỉ giảm; số giá trị $G[i, j]$ mới sinh đóng góp mỗi lần là $O(n)$ và giá trị của chúng cũng không vượt quá $t - i$. Tổng thời gian là $O(n^2)$.

5 1005 Random Walk on Graph

Đặt

$$\lambda_i = W_{i,i}, \quad d_i = \sum_{j=1}^n W_{i,j}.$$

Ta có phương trình chuyển:

$$A_{i,i} = \frac{\lambda_i}{\lambda_i + d_i} + \sum_{\substack{j=1 \\ j \neq i}}^n \frac{W_{i,j}}{\lambda_i + d_i} A_{j,i},$$

và với $i \neq j$,

$$A_{i,j} = \sum_{\substack{k=1 \\ k \neq i}}^n \frac{W_{i,k}}{\lambda_i + d_i} A_{k,j}.$$

Gọi W là ma trận không có khuyên, Λ là ma trận đường chéo của các khuyên, và D là ma trận bậc của W . Khi đó dạng ma trận là

$$(I - (\Lambda + D)^{-1}W)A = (\Lambda + D)^{-1}\Lambda.$$

Các bước là: lập phương trình chuyển, viết thành dạng ma trận, rồi giải hệ ma trận bằng mẫu có sẵn. Sau khi tính được ma trận, chỉ cần dùng phép nghịch đảo ma trận.

6 1006 Cute Tree

Chỉ cần mô phỏng đúng theo đề bài và in số nút của cây.

7 1007 Banzhuan

Với chi phí lớn nhất, rõ ràng cần lấp đầy toàn bộ không gian $n \times n \times n$. Để chi phí lớn nhất, mỗi viên gạch đều được thả từ vị trí cao nhất. Do đó

$$\sum_{x=1}^n \sum_{y=1}^n \sum_{z=1}^n xy^2n = n^2 \cdot \frac{(n+1)n}{2} \cdot \frac{n(n+1)(2n+1)}{6}.$$

Với chi phí nhỏ nhất, do trọng lực, mặt đáy phải được phủ kín. Để thỏa hai hình chiếu còn lại, cần thêm một bức tường đứng tăng theo x và giảm theo y . Vì vậy chi phí nhỏ nhất là

$$\begin{aligned} & \sum_{x=1}^n \sum_{y=1}^n xy + \sum_{y=1}^n \sum_{z=2}^n (n+1-y)y^2z \\ &= \frac{(n+1)^2n^2}{4} + \frac{(n+2)(n-1)}{2} \left(\frac{n(n+1)^2(2n+1)}{6} - \frac{n^2(n+1)^2}{4} \right). \end{aligned}$$

8 1008 Set Counting

Chú ý rằng đề bài yêu cầu một xác suất có điều kiện.

Dùng tiền tố tổng trên siêu lập phương để tính, với mỗi tập T , có bao nhiêu siêu tập U của nó. Gọi giá trị đó là $\text{cnt}[T]$. Khi đó đáp án ban đầu là

$$\sum_{S \in [0, 2^n)} \sum_{T \in [0, 2^n)} \frac{\text{cnt}[T \vee S]}{\text{cnt}[S]}.$$

Biểu thức $T \vee S$ lại có thể được tối ưu bằng một lần tiền tố tổng cao chiều nữa. Gọi kết quả là $\text{cnt0}[T]$. Khi đó

$$\sum_{S \in [0, 2^n)} \frac{\text{cnt0}[S] \cdot 2^{\text{bit}(S)}}{\text{cnt}[S]}.$$

Ở đây $\text{bit}(S)$ là số bit 1 trong S . Độ phức tạp là $O(n2^n)$.

9 1009 Array Counting

Cách thô có độ phức tạp $O(kn \log n)$. Cách dùng cây đoạn theo trọng số hoặc cây Fenwick được lược bỏ trong lời giải gốc.

Cách 3: chia theo số lần xuất hiện. Khi k lớn, chia các giá trị theo số lần xuất hiện: ít nhất $\sqrt{n}$ và nhỏ hơn $\sqrt{n}$. Đếm số lần xuất hiện của mỗi giá trị bằng bucket, bảng băm hoặc tập, rồi sắp theo thứ tự giảm dần.

Nếu một giá trị xuất hiện ít nhất $\sqrt{n}$ lần, số loại như vậy nhiều nhất $\sqrt{n}$. Phần này dùng cách cây đoạn hoặc Fenwick, có độ phức tạp $\sqrt{n} \cdot n \log n$.

Nếu một giá trị xuất hiện ít hơn $\sqrt{n}$ lần, xét các vị trí xuất hiện của nó. Giả sử giá trị đó xuất hiện t_i lần. Liệt kê các lựa chọn l_i, r_i, t_i rồi tính những vị trí hợp lệ ở trái và phải; một giá trị xử lý được trong $O(t_i^2)$. Tổng

$$\sum_i t_i^2 \leq \sqrt{n} \sum_i t_i = n\sqrt{n}.$$

Cách 4: cân bằng hai thuật toán. Đặt ngưỡng chia khối là d . Với các giá trị xuất hiện ít nhất d lần, số loại nhiều nhất n/d , độ phức tạp là

$$\frac{n}{d} \cdot n \log n.$$

Với các giá trị xuất hiện ít hơn d lần, tổng chi phí là nd . Chọn

$$d = \sqrt{n \log n}$$

ta được độ phức tạp

$$O(n^{3/2} \sqrt{\log_2 n}).$$

Cách 5: cây Fenwick. Độ phức tạp là $O(n \log_2 n)$. Đếm số lần xuất hiện của mỗi giá trị, rồi lần lượt xét từng giá trị và hợp nhất ảnh hưởng của hai khối kề nhau cùng giá trị. Đáp án có thể nhìn như: biến mỗi vị trí thành 1 hoặc -1 , lấy tiền tố, rồi đếm trước mỗi tiền tố có bao nhiêu tiền tố nhỏ hơn nó.

Ví dụ mảng ban đầu

$$1, 1, 2, 1$$

được đổi thành

$$1, 1, -1, 1,$$

tiền tố là

$$0, 1, 2, 1, 2,$$

và các đóng góp lần lượt là 1, 2, 1, 3.

Nếu tiền tố hiện tại lớn hơn một tiền tố trước đó, thì đoạn giữa hai tiền tố có tổng dương:

$$\text{sum}[\text{now}] - \text{sum}[\text{last}] > 0.$$

Vì vậy đoạn $\text{last} + 1 \dots \text{now}$ hợp lệ. Phần còn lại là tính đóng góp của từng vị trí 1 đối với các vị trí -1 trước nó. Nếu có k loại, loại thứ i xuất hiện t_i lần, mỗi lần tốn $\log n$, thì tổng độ phức tạp là

$$\sum_i t_i \log n = O(n \log n).$$

Cách 6. Có thể tối ưu cách 5 xuống $O(n)$. Gọi sum là tiền tố hiện tại, $f_1[\text{sum}]$ là số điểm có tiền tố bằng giá trị này. Với mỗi điểm, đóng góp hiện tại là tổng các f_1 ở tiền tố nhỏ hơn sum . Nếu tiếp theo có một đoạn liên tiếp toàn -1 , ta nhảy ngay đến cuối đoạn đó. Dùng thêm một mảng hiệu f_2 : đánh dấu -1 ở đầu đoạn và $+1$ ở cuối đoạn. Khi đi đến i , nếu $f_2[\text{sum}]$ khác 0, dùng nó cập nhật f_1 , đồng thời cập nhật $f_2[\text{sum} + 1]$. Chi tiết cài đặt được lược bỏ.

10 1010 Gumbel Softmax

Với $i = 1, \dots, k$,

$$y_i = \frac{\exp((x_i + g_i - (x_k + g_k)) / \tau)}{\sum_{j=1}^k \exp((x_j + g_j - (x_k + g_k)) / \tau)}.$$

Đặt

$$u_i = x_i + g_i - (x_k + g_k), \quad i = 1, \dots, k-1.$$

Ở đây g_i là biến ngẫu nhiên trong mã giả, tuân theo phân bố Gumbel. Mật độ của phân bố Gumbel là

$$f(z, \mu) = e^{\mu - z - e^{\mu - z}}.$$

Trước hết tính mật độ đồng thời của u :

$$\begin{aligned} p(u_1, \dots, u_{k-1}) &= \int_{-\infty}^{\infty} dg_k p(g_k) \prod_{i=1}^{k-1} p(u_i | g_k) \\ &= \int_{-\infty}^{\infty} dg_k f(g_k, 0) \prod_{i=1}^{k-1} f(x_k + g_k, x_i - u_i). \end{aligned}$$

Sau khi đổi biến và rút gọn, nhận được

$$p(u_1, \dots, u_{k-1}) = \Gamma(k) \left(\prod_{i=1}^k e^{x_i - u_i} \right) \left(\sum_{i=1}^k e^{x_i - u_i} \right)^{-k},$$

trong đó quy ước $u_k = 0$.

Tiếp theo cần mật độ đồng thời của y . Ta có

$$y_k = 1 - \sum_{j=1}^{k-1} y_j,$$

và

$$h_i(u) = \frac{e^{u_i/\tau}}{1 + \sum_{j=1}^{k-1} e^{u_j/\tau}}.$$

Dùng đổi biến:

$$p(y_{1:k}) = p(h^{-1}(y_{1:k-1})) \det \left(\frac{\partial h^{-1}(y_{1:k-1})}{\partial y_{1:k-1}} \right).$$

Nghịch đảo là

$$h_i^{-1}(y) = \tau (\ln y_i - \ln y_k).$$

Jacobian có dạng

$$\tau \left(\text{diag} \left(\frac{1}{y_{1:k-1}} \right) + \frac{1}{y_k} \mathbf{11}^T \right),$$

và định thức là

$$\tau^{k-1} \prod_{j=1}^k y_j^{-1}.$$

Do đó

$$p(y_1, \dots, y_k) = \Gamma(k) \tau^{k-1} \left(\sum_{i=1}^k \frac{e^{x_i}}{y_i^\tau} \right)^{-k} \prod_{i=1}^k \frac{e^{x_i}}{y_i^{\tau+1}}.$$

11 1011 Stone Game

Trường hợp $x = y$. Dễ chứng minh

$$\text{sg}_i = \left\lfloor \frac{i}{2x} \right\rfloor x + i \bmod x.$$

Khi đó bài toán trở thành Nim: nếu

$$\bigoplus_{i=1}^n \text{sg}_{a_i} > 0$$

thì người đi trước thắng, ngược lại thua.

Sau đây đặt $\text{sg}_i = i$, tức giá trị SG của Nim.

Trường hợp $x < y$. Giả sử

$$a_1 \leq a_2 \leq \dots \leq a_n.$$

Nếu $a_n < x$, bài toán là Nim, nên chỉ cần xét $a_n \geq x$.

Nếu người đi trước có thể qua một thao tác làm cho

$$\max_i a_i < x,$$

thì thắng thua do $\bigoplus_i a_i$ quyết định. Nếu tìm được số dương z sao cho

$$\bigoplus_{i=1}^n a_i \oplus a_n \oplus (a_n - z) = 0,$$

thì người đi trước thắng; nếu không, người đi trước phải xét chiến lược khác.

Khi người đi trước không thể đưa mọi đồng xuống dưới x trong một thao tác, phân tích trạng thái sau lượt đầu theo ba trường hợp.

1. $a_{n-1} < x$ và $a_n \geq x$. Nếu xor hiện tại khác 0, tồn tại một đồng j và số dương z sao cho

$$\bigoplus_i a_i \oplus a_j \oplus (a_j - z) = 0.$$

Nếu $z \neq y$, người đi sau trực tiếp lấy z viên để đưa người đi trước vào trạng thái xor bằng 0. Nếu $z = y$, khi đó $j = n$; người đi sau lấy $y - x$ viên từ a_n , khiến người đi trước không thể lấy x viên từ đồng thứ n .

Nếu xor hiện tại bằng 0, giả sử người đi sau lấy x viên từ a_n . Theo tính chất của Nim và $a_{n-1} < x < y$, người đi trước chắc chắn có nước đáp trả. Người đi sau chọn đúng nước đáp trả đó để khiến người đi trước rơi vào tình huống không thể lấy x viên từ đồng thứ n .

Vì vậy mọi cục diện người đi trước có thể tạo ra đều là

$$\bigoplus_i a_i = 0 \quad \text{hoặc} \quad \bigoplus_i a_i \oplus a_n \oplus (a_n - x) = 0,$$

và người đi sau thắng.

2. $a_{n-2} < x$, $a_{n-1} \geq x$, $a_n \geq x$. Người đi trước sẽ không tự lấy để làm $a_{n-1} < x$ hoặc $a_n < x$, và người đi sau cũng vậy. Cuối cùng còn hai đồng $a_{n-1} = x$ và $a_n = x$. Nếu đến lượt người đi trước, người đi sau đối xứng; nếu đến lượt người đi sau, người đi sau lấy hết một đồng x , còn lại một đồng x mà người đi trước không thể lấy hết trong một lượt. Người đi sau thắng.

3. $a_{n-2} \geq x$, $a_{n-1} \geq x$, $a_n \geq x$. Người đi sau chỉ cần sau mỗi lượt vẫn duy trì trường hợp này, buộc người đi trước tự đi vào trường hợp 2. Người đi trước sẽ không chủ động vào trường hợp 2; cuối cùng còn ba đồng x , người đi sau lấy hết một đồng rồi đối xứng.

Tóm lại, khi $x < y$, người đi trước thắng khi có thể qua một thao tác làm $\max_i a_i < x$ và thỏa điều kiện xor ở trên; nếu không thì thua.

Trường hợp $x > y$. Nếu $a_n < y$, bài toán là Nim. Nếu $a_n \geq y$, tình huống đối ứng với ba trường hợp đã xét khi $x < y$, và người đi trước thắng.

12 1012 Matrix Counting

Chú ý

$$\begin{aligned} \text{ans}[i] &= \left[\sum_{i=1}^n \sum_{j=1}^r S_{ij} = i \right] \\ &= \left[\sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^r A_{ik} B_{kj} = i \right] \\ &= \left[\sum_{k=1}^r \left(\sum_{i=1}^n A_{ik} \right) \left(\sum_{j=1}^n B_{kj} \right) = i \right]. \end{aligned}$$

Gọi $F[q]$ là số phương án để

$$\sum_{i=1}^n A_{ik} = q,$$

và $G[q]$ tương tự cho B . Rõ ràng $F[q] = G[q]$.

Để tính $F[x]$, xét hàm sinh

$$g(x) = (1 + x + x^2 + \dots + x^m)^n,$$

khi đó

$$F[q] = [x^q]g(x).$$

Có hai cách tính. Với $q = 0, 1, \dots, m$, hệ số bậc q tương đương số nghiệm nguyên không âm của

$$x_1 + x_2 + \dots + x_n = q,$$

nên bằng

$$[x^q]g(x) = \binom{n+q-1}{n-1}.$$

Ngoài ra có thể dùng lũy thừa đa thức nhanh bằng cách lấy ln rồi exp.

Gọi $H[i]$ là số phương án để

$$\left(\sum_i A_{ik}\right)\left(\sum_j B_{kj}\right) = i.$$

Ta có thể liệt kê x, y và cộng

$$H[xy] += F[x]G[y]$$

để thu được các giá trị $H[i]$ với $i = 0, 1, \dots, m$.

Lưu ý rằng các giá trị như

$$H[0] += F[0]G[y], \quad y = m+1, \dots, mn$$

chưa được tính trong phép liệt kê trên. Do

$$\sum_{i=0}^{mn} F[i] = \sum_{i=0}^{mn} G[i] = (m+1)^n, \quad F[0] = G[0] = 1,$$

cần hiệu chỉnh

$$H[0] = (m+1)^n - 1.$$

Đặt hàm sinh

$$h(x) = (H[0] + H[1]x + H[2]x^2 + \dots + H[m]x^m)^r.$$

Khi đó

$$\text{ans}[i] = [x^i]h(x).$$

Ta dùng lũy thừa đa thức nhanh để tính $h(x)$. Vì r rất lớn, dùng định lý Euler để xử lý số mũ. Độ phức tạp là $O(m \log m)$.

13 1013 Tree Diameter

Bài toán tương đương với: sau khi thao tác trên n điểm, làm cho đường kính của cây ngắn nhất có thể.

Nhiệm vụ phân đáp án. Mỗi lần kiểm tra, dùng giá trị giữa làm ngưỡng để hạn chế chuyển trạng thái trong quy hoạch động trên cây.

Với cây con gốc u , duy trì hai trạng thái. $dp[u][0]$ biểu diễn trường hợp giá trị power của u được cho con; khi đó nó lưu độ dài chuỗi dài nhất từ một lá trong cây con của u đến u . $dp[u][1]$ biểu diễn trường hợp power của u được cho cha, cũng lưu độ dài chuỗi dài nhất từ lá đến u .

Với chuyển trạng thái của $dp[u][1]$, chỉ cần bảo đảm mọi cặp con đi qua u đều có tổng độ dài không vượt ngưỡng, rồi ghi lại chuỗi dài nhất. Với $dp[u][0]$, liệt kê việc power của u đóng góp cho chuỗi dài nhất hoặc chuỗi dài thứ hai; trong số các lựa chọn thỏa mọi tổng độ dài qua hai con không vượt ngưỡng, lấy chuỗi dài nhất nhỏ nhất. Cuối cùng chỉ cần kiểm tra trạng thái tại gốc có hợp lệ hay không.

Độ phức tạp là

$$O(n \log(n \cdot \max w)).$$